

WEEK 3 ASSIGNMENT

SQL Mini Project: Sales Analysis using Subqueries, CTEs, and Window Functions

Objective:

The objective of this project is to analyze sales data from the Superstore dataset using SQL.

This project uses **Subqueries, Common Table Expressions (CTEs), and Window Functions** to solve business queries such as above-average sales, highest sales per customer, customer ranking, and customer sales insights.

Tools Used

- MySQL Workbench
- SQL
- Superstore Dataset

Step 1: Setup Data

1. Import the Superstore dataset into a table named `superstore_raw`.

- The Superstore dataset was imported into a raw table named `superstore_raw` in MySQL Workbench.
- The dataset contained sales-related information such as order details, customer details, product details, sales, quantity, discount, and profit.

Import Results

File C:\Users\HP\Desktop\celebal_technology\assignment_3\superstore_raw.csv was imported in 77.614 s

Table superstore_db.superstore_raw was created

9694 records imported

✓	3	13:40:55	SHOW SESSION VARIABLES LIKE 'lower_case_table_names'	OK	0.000 sec
✓	4	13:40:55	SHOW DATABASES	OK	0.000 sec
✓	5	13:41:28	SHOW SESSION VARIABLES LIKE 'lower_case_table_names'	OK	0.000 sec
✓	6	13:41:28	SHOW TABLES FROM 'superstore_db' like 'superstore_raw'	OK	0.000 sec
✓	7	13:41:44	CREATE TABLE 'superstore_db'.'superstore_raw' ('Row ID' int, 'Order ID' text, 'Order...	OK	0.000 sec
✓	8	13:41:44	PREPARE stmt FROM 'INSERT INTO 'superstore_db'.'superstore_raw' ('Row ID','Or...	OK	0.000 sec
✓	9	13:43:01	DEALLOCATE PREPARE stmt	OK	0.000 sec

2. Create database

```
CREATE DATABASE superstore_db;
```

```
USE superstore_db;
```

#	Time	Action	Message	Duration / Fetch
✓	1	13:30:05	CREATE DATABASE superstore_db	1 row(s) affected 0.015 sec
✓	2	13:30:08	USE superstore_db	0 row(s) affected 0.000 sec

3. Create these 3 tables from it:

- customers
- orders
- products

```
DROP TABLE IF EXISTS orders;
```

```
DROP TABLE IF EXISTS products;
```

```
DROP TABLE IF EXISTS customers;
```

```
-- Customers table
```

```
CREATE TABLE customers (
    customer_id VARCHAR(50) PRIMARY KEY,
    customer_name VARCHAR(100),
    segment VARCHAR(50),
    country VARCHAR(50),
    city VARCHAR(50),
    state VARCHAR(50),
```

```
    postal_code VARCHAR(20),  
    region VARCHAR(50)  
);
```

-- Products table

```
CREATE TABLE products (  
    product_id VARCHAR(50) PRIMARY KEY,  
    category VARCHAR(50),  
    sub_category VARCHAR(50),  
    product_name VARCHAR(255)  
);
```

-- Orders table

```
CREATE TABLE orders (  
    row_id INT PRIMARY KEY,  
    order_id VARCHAR(50),  
    order_date VARCHAR(20),  
    ship_date VARCHAR(20),  
    ship_mode VARCHAR(50),  
    customer_id VARCHAR(50),  
    product_id VARCHAR(50),  
    sales DECIMAL(10,2),  
    quantity INT,  
    discount DECIMAL(5,2),
```

profit DECIMAL(10,2)

);

⚠	12	13:54:58	DROP TABLE IF EXISTS orders	0 row(s) affected, 1 warning(s): 1051 Unknown table 'superstore_db.orders'	0.016 sec
⚠	13	13:55:01	DROP TABLE IF EXISTS products	0 row(s) affected, 1 warning(s): 1051 Unknown table 'superstore_db.products'	0.031 sec
⚠	14	13:55:03	DROP TABLE IF EXISTS products	0 row(s) affected, 1 warning(s): 1051 Unknown table 'superstore_db.products'	0.031 sec
⚠	15	13:55:06	DROP TABLE IF EXISTS customers	0 row(s) affected, 1 warning(s): 1051 Unknown table 'superstore_db.customers'	0.031 sec
✅	16	13:55:25	CREATE TABLE customers (customer_id VARCHAR(50) PRIMARY KEY, cus...	0 row(s) affected	0.063 sec
✅	17	13:55:31	CREATE TABLE products (product_id VARCHAR(50) PRIMARY KEY, categ...	0 row(s) affected	0.078 sec
✅	18	13:55:35	CREATE TABLE orders (row_id INT PRIMARY KEY, order_id VARCHAR(50),...	0 row(s) affected	0.063 sec

4. Insert data into these tables using SELECT DISTINCT.

INSERT INTO customers (

customer_id,

customer_name,

segment,

country,

city,

state,

postal_code,

region

)

SELECT

`Customer ID` AS customer_id,

MAX(`Customer Name`) AS customer_name,

MAX(Segment) AS segment,

MAX(Country) AS country,

MAX(City) AS city,

MAX(State) AS state,

MAX(`Postal Code`) AS postal_code,

MAX(Region) AS region

FROM superstore_raw

GROUP BY `Customer ID`;

INSERT INTO products (

product_id,

category,

sub_category,

product_name

```
)  
SELECT  
    `Product ID` AS product_id,  
    MAX(Category) AS category,  
    MAX(`Sub-Category`) AS sub_category,  
    MAX(`Product Name`) AS product_name  
FROM superstore_raw  
GROUP BY `Product ID`;
```

```
INSERT INTO orders (
```

```
    row_id,  
    order_id,  
    order_date,  
    ship_date,  
    ship_mode,  
    customer_id,  
    product_id,  
    sales,  
    quantity,  
    discount,  
    profit
```

```
)
```

```
SELECT DISTINCT
```

```
    `Row ID`,  
    `Order ID`,  
    `Order Date`,  
    `Ship Date`,  
    `Ship Mode`,  
    `Customer ID`,  
    `Product ID`,  
    Sales,  
    Quantity,  
    Discount,  
    Profit
```

```
FROM superstore_raw;
```

✓	25	14:02:16	INSERT INTO customers (customer_id, customer_name, segment, count...	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0	0.093 sec
✓	26	14:02:30	INSERT INTO products (product_id, category, sub_category, product_n...	1812 row(s) affected Records: 1812 Duplicates: 0 Warnings: 0	0.156 sec
⚠	27	14:03:01	INSERT INTO orders (row_id, order_id, order_date, ship_date, ship_m...		0.328 sec

Step 2: Perform Required Queries

1. Find all orders where sales are greater than the average sales. (Subquery)

```
SELECT *
FROM orders
WHERE sales > (
    SELECT AVG(sales)
    FROM orders
);
```

row_id	order_id	order_date	ship_date	ship_mode	customer_id	product_id	sales
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-BO-10001798	261.96
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.94
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.58
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002275	907.15
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.18
12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002033	911.42
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	OFF-BI-10003656	407.98
17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	OFF-ST-10004186	665.88

orders 1 x

33 14:10:23 SELECT * FROM orders WHERE sales > (SELECT AVG(sales) FROM orders ... 1000 row(s) returned 0.016 sec / 0.000 sec

2. Find the highest sales order for each customer.

```
WITH max_sales_per_customer AS (
    SELECT
        customer_id,
        MAX(sales) AS max_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    o.customer_id,
```

```

        c.customer_name,
        o.order_id,
        o.sales
FROM orders o
JOIN max_sales_per_customer m
    ON o.customer_id = m.customer_id
    AND o.sales = m.max_sales
JOIN customers c
    ON o.customer_id = c.customer_id
ORDER BY c.customer_name;

```

	customer_id	customer_name	order_id	sales
▶	AB-10015	Aaron Bergman	CA-2016-140935	341.96
	AH-10030	Aaron Hawkins	CA-2015-130113	668.16
	AS-10045	Aaron Smayling	CA-2017-162691	1439.98
	AB-10060	Adam Bellavance	CA-2016-129714	4355.17
	AH-10075	Adam Hart	CA-2016-149797	841.57
	AS-10090	Adam Shillingsburg	CA-2015-127509	1024.38
	AB-10105	Adrian Barton	CA-2016-117121	9892.74

Result 2 x

37 14:17:07 WITH max_sales_per_customer AS (SELECT customer_id, MAX(sales... 795 row(s) returned

3. Calculate total sales for each customer. (CTE)

```

WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    cs.customer_id,
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id

```

ORDER BY cs.total_sales DESC;

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	customer_id	customer_name	total_sales			
▶	SM-20320	Sean Miller	25043.07			
	TC-20980	Tamara Chand	19017.85			
	RB-19360	Raymond Buch	15117.35			
	TA-21385	Tom Ashbrook	14595.62			
	AB-10105	Adrian Barton	14355.61			

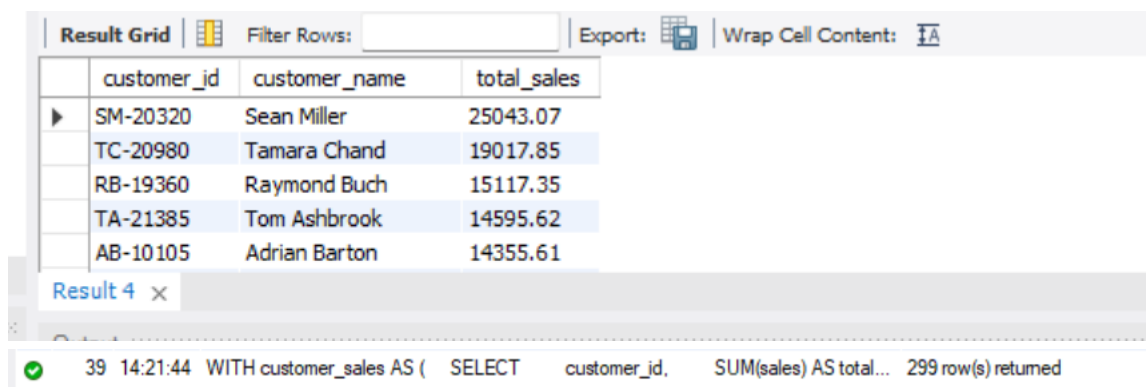
Result 3 x

38 14:20:32 WITH customer_sales AS (SELECT customer_id, SUM(sales) AS total... 793 row(s) returned

4. Find customers whose total sales are above average. (CTE + Subquery)

```
WITH customer_sales AS (  
  SELECT  
    customer_id,  
    SUM(sales) AS total_sales  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT  
  cs.customer_id,  
  c.customer_name,  
  cs.total_sales  
FROM customer_sales cs  
JOIN customers c  
  ON cs.customer_id = c.customer_id  
WHERE cs.total_sales > (  
  SELECT AVG(total_sales)  
  FROM customer_sales  
)
```

ORDER BY cs.total_sales DESC;



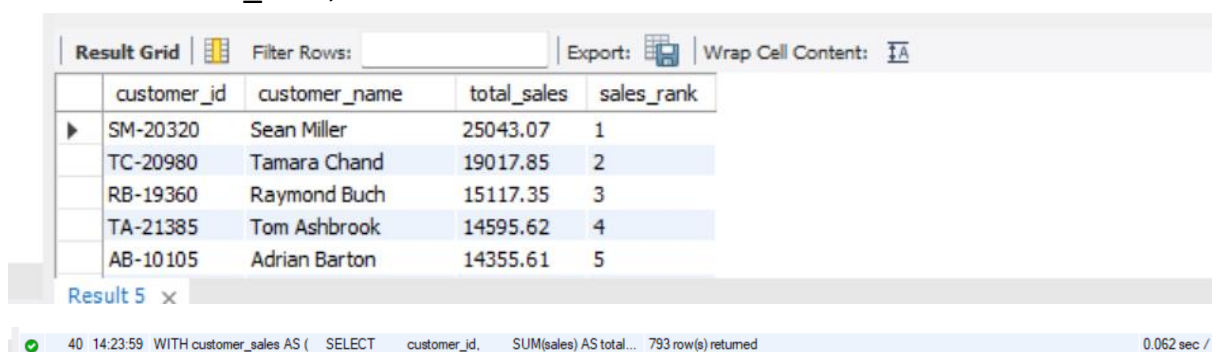
	customer_id	customer_name	total_sales
▶	SM-20320	Sean Miller	25043.07
	TC-20980	Tamara Chand	19017.85
	RB-19360	Raymond Buch	15117.35
	TA-21385	Tom Ashbrook	14595.62
	AB-10105	Adrian Barton	14355.61

Result 4 x

39 14:21:44 WITH customer_sales AS (SELECT customer_id, SUM(sales) AS total... 299 row(s) returned

5. Rank all customers based on total sales. (*Window Function*)

```
WITH customer_sales AS (  
    SELECT  
        customer_id,  
        SUM(sales) AS total_sales  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT  
    cs.customer_id,  
    c.customer_name,  
    cs.total_sales,  
    RANK() OVER (ORDER BY cs.total_sales DESC) AS sales_rank  
FROM customer_sales cs  
JOIN customers c  
    ON cs.customer_id = c.customer_id  
ORDER BY sales_rank;
```



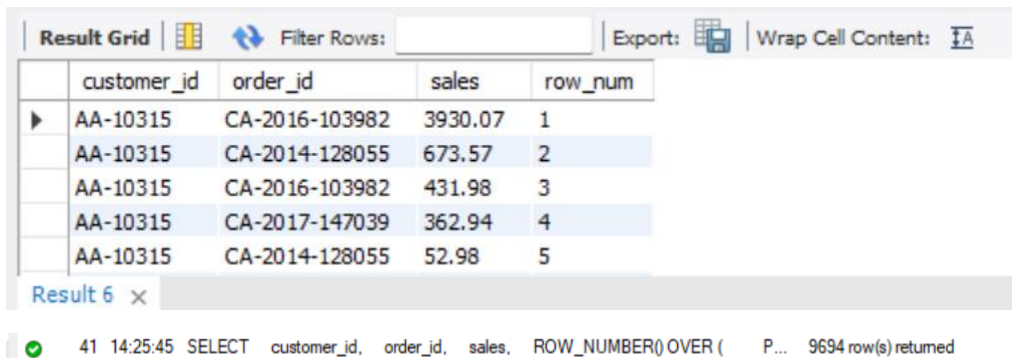
	customer_id	customer_name	total_sales	sales_rank
▶	SM-20320	Sean Miller	25043.07	1
	TC-20980	Tamara Chand	19017.85	2
	RB-19360	Raymond Buch	15117.35	3
	TA-21385	Tom Ashbrook	14595.62	4
	AB-10105	Adrian Barton	14355.61	5

Result 5 x

40 14:23:59 WITH customer_sales AS (SELECT customer_id, SUM(sales) AS total... 793 row(s) returned 0.062 sec /

6. Assign row numbers to each order within a customer. (*Window Function + PARTITION BY*)

```
SELECT
    customer_id,
    order_id,
    sales,
    ROW_NUMBER() OVER (
        PARTITION BY customer_id
        ORDER BY sales DESC
    ) AS row_num
FROM orders;
```



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays the results of the SQL query, with columns: customer_id, order_id, sales, and row_num. The data is sorted by customer_id and then by sales in descending order. The first customer, AA-10315, has five orders with row numbers 1 through 5. The interface includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' option. Below the grid, a status bar indicates 'Result 6' and a message: '41 14:25:45 SELECT customer_id, order_id, sales, ROW_NUMBER() OVER (P... 9694 row(s) returned'.

	customer_id	order_id	sales	row_num
▶	AA-10315	CA-2016-103982	3930.07	1
	AA-10315	CA-2014-128055	673.57	2
	AA-10315	CA-2016-103982	431.98	3
	AA-10315	CA-2017-147039	362.94	4
	AA-10315	CA-2014-128055	52.98	5

7. Display top 3 customers based on total sales. (*Window Function*)

```
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
),
ranked_customers AS (
    SELECT
        cs.customer_id,
        c.customer_name,
        cs.total_sales,
        RANK() OVER (ORDER BY cs.total_sales DESC) AS sales_rank
```

```

FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id
)
SELECT *
FROM ranked_customers
WHERE sales_rank <= 3
ORDER BY sales_rank;

```

Result Grid				
Filter Rows:		Export:		
	customer_id	customer_name	total_sales	sales_rank
▶	SM-20320	Sean Miller	25043.07	1
	TC-20980	Tamara Chand	19017.85	2
	RB-19360	Raymond Buch	15117.35	3

42 14:27:48 WITH customer_sales AS (SELECT customer_id, SUM(sales) AS total... 3 row(s) returned 0.016 sec / 0.000 sec

Step 3: Final Combined Query

Write **one final query** that shows:

- Customer Name
- Total Sales
- Rank

(Use **JOIN + CTE + Window Function together**)

```

WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

```

SELECT

c.customer_name,

cs.total_sales,

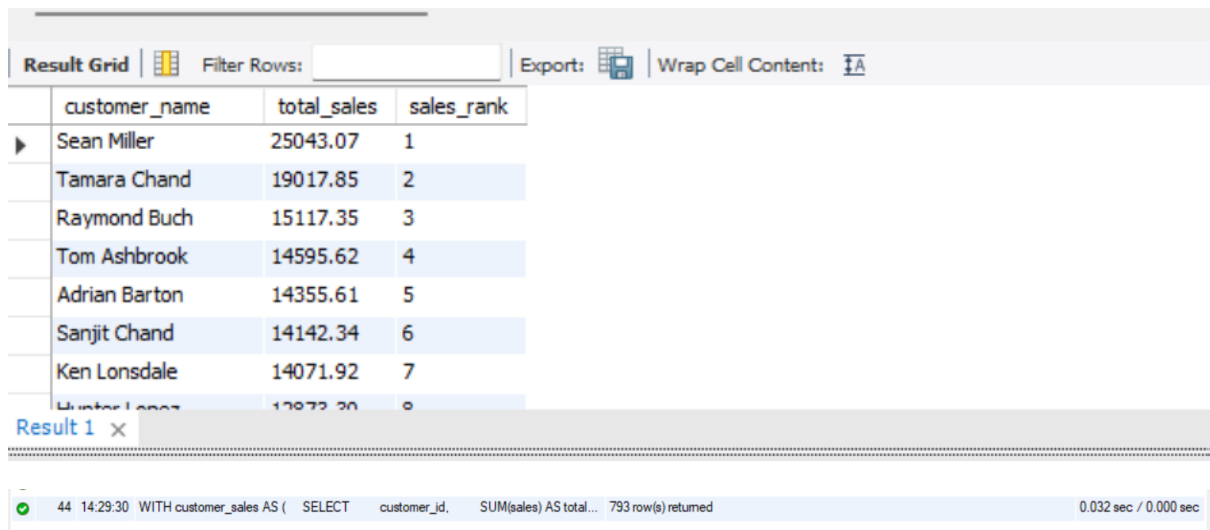
RANK() OVER (ORDER BY cs.total_sales DESC) AS sales_rank

FROM customer_sales cs

JOIN customers c

ON cs.customer_id = c.customer_id

ORDER BY sales_rank;



The screenshot shows a SQL query result grid with columns: customer_name, total_sales, and sales_rank. The results are ordered by total_sales in descending order. The top 8 results are visible. Below the grid, a status bar indicates the query was executed at 14:29:30, returned 793 rows, and took 0.032 seconds.

customer_name	total_sales	sales_rank
Sean Miller	25043.07	1
Tamara Chand	19017.85	2
Raymond Buch	15117.35	3
Tom Ashbrook	14595.62	4
Adrian Barton	14355.61	5
Sanjit Chand	14142.34	6
Ken Lonsdale	14071.92	7
Hunter Jones	13872.30	8

Result 1 x

44 14:29:30 WITH customer_sales AS (SELECT customer_id, SUM(sales) AS total... 793 row(s) returned 0.032 sec / 0.000 sec

This query combines **JOIN**, **CTE**, and **Window Function** to display each customer's name, total sales, and sales rank in a single result.

Mini Project: Customer Sales Insights

1. Who are the top 5 customers?

WITH customer_sales AS (

SELECT

customer_id,

SUM(sales) AS total_sales

FROM orders

```

        GROUP BY customer_id
    )
SELECT
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id
ORDER BY cs.total_sales DESC
LIMIT 5;

```

	customer_name	total_sales
▶	Sean Miller	25043.07
	Tamara Chand	19017.85
	Raymond Buch	15117.35
	Tom Ashbrook	14595.62
	Adrian Barton	14355.61

The top 5 customers contribute the highest share of sales and represent the most valuable customers for the business.

2. Who are the bottom 5 customers?

```

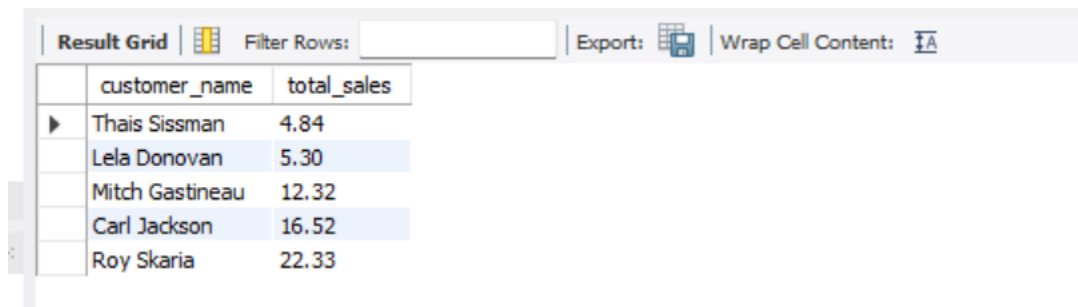
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)

```

```

SELECT
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id
ORDER BY cs.total_sales ASC
LIMIT 5;

```



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with two columns: 'customer_name' and 'total_sales'. The table contains five rows of data, which are the bottom 5 customers by total sales. The interface also includes a 'Filter Rows' search bar, an 'Export' button, and a 'Wrap Cell Content' toggle.

	customer_name	total_sales
▶	Thais Sissman	4.84
	Lela Donovan	5.30
	Mitch Gastineau	12.32
	Carl Jackson	16.52
	Roy Skaria	22.33

The bottom 5 customers contribute very little sales and may need re-engagement or promotional strategies.

3. Which customers made only one order?

```

SELECT
    c.customer_id,
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(DISTINCT o.order_id) = 1;

```

Result Grid			
		Filter Rows:	
		Export:	Wrap Cell
	customer_id	customer_name	total_orders
▶	AO-10810	Anthony O'Donnell	1
	AR-10570	Anemone Ratner	1
	CJ-11875	Carl Jackson	1
	JC-15385	Jenna Caffey	1
	JR-15700	Jocasta Rupert	1
	LD-16855	Lela Donovan	1
	MG-18205	Mitch Gastineau	1
	PH-18790	Patricia Hirasaki	1
	RE-19405	Ricardo Emerson	1
	RM-19750	Roland Murray	1
	SM-20905	Susan MacKendrick	1
	TC-21145	Theresa Coyne	1

Customers with **only one order** can be studied for retention and repeat-purchase analysis.

4. Which customers have above-average sales?

```

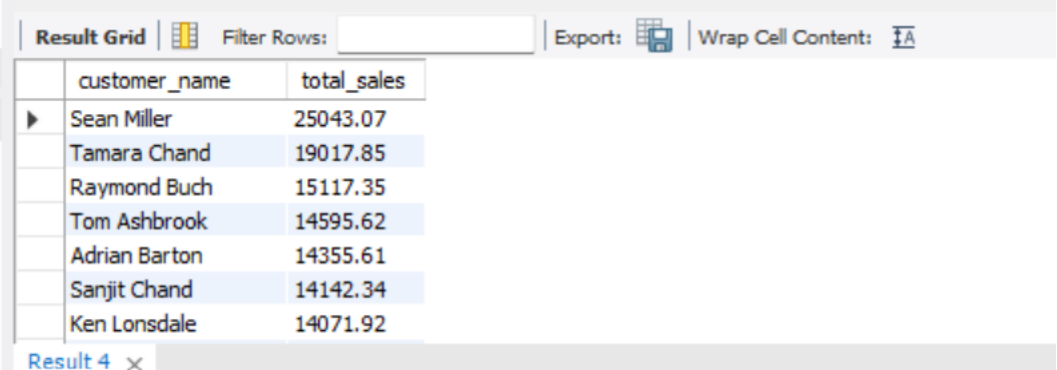
WITH customer_sales AS (
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.customer_name,
    cs.total_sales
FROM customer_sales cs
JOIN customers c
    ON cs.customer_id = c.customer_id

```

```

WHERE cs.total_sales > (
    SELECT AVG(total_sales)
    FROM customer_sales
)
ORDER BY cs.total_sales DESC;

```



The screenshot shows a SQL query result grid with two columns: 'customer_name' and 'total_sales'. The results are ordered by total sales in descending order. The top customer is Sean Miller with a total sales of 25043.07. The bottom customer shown is Ken Lonsdale with a total sales of 14071.92. The grid includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' option.

customer_name	total_sales
Sean Miller	25043.07
Tamara Chand	19017.85
Raymond Buch	15117.35
Tom Ashbrook	14595.62
Adrian Barton	14355.61
Sanjit Chand	14142.34
Ken Lonsdale	14071.92

Customers with above-average sales form a high-performing segment and can be targeted with loyalty benefits.

5. What is the highest order value per customer?

```

WITH max_sales_per_customer AS (
    SELECT
        customer_id,
        MAX(sales) AS max_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    o.customer_id,
    c.customer_name,
    o.order_id,

```

```

o.sales AS highest_order_value
FROM orders o
JOIN max_sales_per_customer m
    ON o.customer_id = m.customer_id
    AND o.sales = m.max_sales
JOIN customers c
    ON o.customer_id = c.customer_id
ORDER BY c.customer_name;

```

Result Grid				
Filter Rows:		Export:		
		Wrap Cell Content:		
	customer_id	customer_name	order_id	highest_order_value
▶	AB-10015	Aaron Bergman	CA-2016-140935	341.96
	AH-10030	Aaron Hawkins	CA-2015-130113	668.16
	AS-10045	Aaron Smayling	CA-2017-162691	1439.98
	AB-10060	Adam Bellavance	CA-2016-129714	4355.17
	AH-10075	Adam Hart	CA-2016-149797	841.57
	AS-10090	Adam Shillingsburg	CA-2015-127509	1024.38
	AB-10105	Adrian Barton	CA-2016-117121	9892.74

The highest order value per customer helps identify customers with strong purchase capacity or premium buying behavior.

Insights

1. The analysis shows that a **small group of customers contributes a large portion of the total sales**, indicating the presence of high-value customers.
2. Customers whose **total sales are above average** represent an important customer segment for the business and can be targeted for loyalty or retention strategies.
3. The **top 5 customers** contribute significantly more sales than the bottom customers, highlighting sales concentration among a few customers.
4. Customers who made **only one order** are important for retention analysis because they may not have returned after their first purchase.
5. Using **window functions** such as RANK() and ROW_NUMBER() made it easier to rank customers and analyze order-level performance.
6. The **highest order value per customer** helps identify customers with stronger purchase capacity and premium buying behavior.

Conclusion

This project successfully analyzed the Superstore sales dataset using SQL in MySQL Workbench.

The dataset was first imported into a raw table and then organized into separate **customers, products, and orders** tables. Using **Subqueries, CTEs, and Window Functions**, several business questions were answered, such as identifying orders above average sales, calculating total customer sales, ranking customers, and finding top-performing customers.

The project also provided customer-level insights through the mini project section, including top and bottom customers, one-time customers, and customers with above-average sales. Overall, this project demonstrates how SQL can be used effectively for **data analysis, business reporting, and extracting customer sales insights** from a retail dataset.

